

ROCm 双 GPU 配置方案 — 方法与工具包

Copyright 2026 cubecloud Limited (<https://cubecloud.io>) · 基于 [Apache License 2.0](#) 许可

本工具包复现了 **Option 2** 双 GPU 模式，适用于：

- **iGPU** (如 Strix Halo gfx1151、Strix Point gfx1155、Phoenix gfx1103、Rembrandt gfx1103、Van Gogh gfx1035 等) — 使用 **TheRock pip wheels**
- **dGPU** (如 Navi 31/32/33 = gfx1100/1101/1102; RDNA 4 = gfx1200/1201) — 使用 **系统 HIP SDK**

方案在以下环境中开发验证： - AMD Adrenalin 驱动 32.0.31019.2002 (Windows) - AMD HIP SDK 7.1.0 (C:\Program Files\AMD\ROCm\7.1\) - TheRock Python 轮子包，源

<https://repo.amd.com/rocm/whl/<target>/> (本方案写作时为 7.13.0) - Python 3.12.10 (系统级) - Visual Studio 2022 BuildTools (C++ 工作负载, Windows SDK 10.0.26100)

磁盘预算：每个 TheRock 虚拟环境约 22 GB，HIP SDK 约 3 GB。dGPU 侧复用 HIP SDK，iGPU 侧运行在虚拟环境中。

再次使用本工具包时，版本可能已经更新 — <https://repo.amd.com/rocm/whl/<target>/> 的轮子包会变化。下方方法是版本无关的；示例命令中的 URL 仅作参考。

方法（共六个阶段）

阶段 0 — 准备工作

1. **驱动：**安装 AMD Adrenalin PRO / Adrenalin (最新 WHQL)。它会向用户态暴露两块 GPU 并提供 `hipInfo` 数据。
2. **HIP SDK 7.1.0** (或当前版本)：安装全部 8 个组件。默认安装路径 `C:\Program Files\AMD\ROCm\7.1\`。
3. **Python 3.12.x** (系统级，例如 `C:\Program Files\Python312`)。
4. **VS 2022 BuildTools** (C++ 工作负载 + Windows SDK)，路径 `C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools\`。仅在需要为 dGPU 编译 C++ 时必需。
5. 运行 `hipInfo.exe` (位于 `C:\Program\AMD\ROCm\7.1\bin\`)，**记录**每块设备对应的 `gcnArchName`。这就是你 dGPU 的目标架构。例如：

```
device#0 Name: AMD Radeon(TM) 8060S Graphics gcnArchName: gfx1151 87.87 GB <- Strix Halo iGPU device#1  
Name: AMD Radeon RX 7900 XTX gcnArchName: gfx1100 23.98 GB <- RDNA3 dGPU
```

驱动会同时看到两块 GPU；其中一块是你的 iGPU 目标，另一块是 dGPU 目标。

阶段 1 — iGPU 虚拟环境 (TheRock pip 轮子包)

本阶段配置 iGPU 侧。目标 family 来自 iGPU 的 `gcnArchName`。将你的 iGPU 映射到正确的 TheRock 索引：

iGPU	架构	TheRock 索引
Strix Halo (Ryzen AI MAX 395)	gfx1151	gfx1151
Strix Point (Ryzen AI HX 370/470)	gfx1155	gfx1151 或 gfx12-generic (视 AMD 发布情况)
Phoenix (Ryzen 7040HS)	gfx1103	gfx110X-all (更宽泛)
Rembrandt (Ryzen 6000)	gfx1103	gfx110X-all
Van Gogh (Steam Deck)	gfx1035	gfx103X-all

如有疑问，可访问 <https://repo.amd.com/rocm/whl/> 查看 AMD 发布了哪些目标。

```
$ErrorActionPreference = 'Stop'
$INDEX = 'https://repo.amd.com/rocm/whl/gfx1151/'      # <-- 设为你的 iGPU 目标
$VENV = 'C:\rocm-sdk'
$PY    = 'C:\Program Files\Python312\python.exe'

# 1. 创建虚拟环境
& $PY -m venv $VENV\venv
& "$VENV\venv\Scripts\python.exe" -m pip install --upgrade pip wheel setuptools

# 2. 预下载轮子包到本地缓存（之后可离线安装）
$cache = "$VENV\cache"
New-Item -ItemType Directory -Force -Path $cache | Out-Null
& $VENV\venv\Scripts\python.exe -m pip download --no-deps --no-build-isolation `
    --index-url "$INDEX" --dest $cache `
    rocm_bootstrap rocm-sdk-core rocm-sdk-devel rocm-sdk-libraries-<你的目标> rocm

# 3. 从本地缓存安装（无需联网，避免找不到包的问题）
& $VENV\venv\Scripts\python.exe -m pip install --no-index --find-links $cache --no-build-isolation -
    rocm_bootstrap==0.1.0 rocm-sdk-core==<版本> rocm-sdk-devel=<版本> rocm-sdk-libraries-<你的目标>==<版
& $VENV\venv\Scripts\python.exe -m pip install --no-index --find-links $cache --no-build-isolation -

# 4. 冒烟测试
& $VENV\venv\Scripts\python.exe -m rocm_sdk version          # <版本>
& $VENV\venv\Scripts\python.exe -m rocm_sdk targets          # 你的 iGPU 架构
& $VENV\venv\Scripts\python.exe -m rocm_sdk test              # 26/26 (1 个 Linux 跳过)
& $VENV\venv\bin\hipInfo.exe                                  # 列出两块 GPU: iGPU 的 gcnArch 应匹配
```

关键安装顺序：先装 3 个轮子包，最后装 `rocm` 元 sdist。元包提供可导入的 `rocm_sdk` 模块 — 没有它，`python -m rocm_sdk` 会报 "No module named rocm_sdk"。

阶段 2 — iGPU 环境重写（机器范围，需 UAC）

```
# 备份当前环境
[Environment]::GetEnvironmentVariables('User') | Export-Clixml "$VENV\env-backup.xml"
[Environment]::GetEnvironmentVariables('Machine') | Export-Clixml "$VENV\env-backup-machine.xml"
```

```
# 用户范围
[Environment]::SetEnvironmentVariable('HIP_PATH', "$VENV\.venv\Lib\site-packages\_rocm_sdk_core", 'U
[Environment]::SetEnvironmentVariable('LLVM_PATH', "$VENV\.venv\Lib\site-packages\_rocm_sdk_devel\lib
$userPath = [Environment]::GetEnvironmentVariable('PATH', 'User')
$prepend = "$VENV\.venv\Scripts;$VENV\.venv\Lib\site-packages\_rocm_sdk_core\bin;$VENV\.venv\Lib\site
[Environment]::SetEnvironmentVariable('PATH', $prepend + ';' + $userPath, 'User')

# 机器范围 (UAC)
$msiPath = [Environment]::GetEnvironmentVariable('PATH', 'Machine')
Start-Process powershell -ArgumentList '-NoProfile', '-Command', `
    "[Environment]::SetEnvironmentVariable('HIP_PATH', '$VENV\.venv\Lib\site-packages\_rocm_sdk_core',
    "[Environment]::SetEnvironmentVariable('PATH', '$VENV\.venv\Scripts;$VENV\.venv\Lib\site-packages\
    -Verb RunAs -Wait

# 在新 shell 中验证
Start-Process pwsh -ArgumentList '-NoProfile', '-Command', 'Get-Command hipconfig,hipcc,rocm-sdk,clang
```

阶段 3 — dGPU 环境 (HIP SDK, 按需激活)

dGPU 侧**不需要**单独的虚拟环境。系统级 HIP SDK 7.1.0 已经能识别 dGPU。激活脚本只是清掉 iGPU 虚拟环境对 `HIP_PATH` 的覆盖，并把 HIP SDK 的 `bin/lib` 加到 `PATH` 前面。

将本工具包中的 `activate-dgpu.ps1` 和 `deactivate-dgpu.ps1` 放到任意目录（如 `C:\rocm-sdk-dgpu\`）。它们在激活时给当前环境拍快照，停用时恢复。

阶段 4 — dGPU C++ 编译 (按需)

两种方式：

方式 A：运行已编译的 .exe（内核已编译好）。只需 `activate-dgpu.ps1` 然后 `HIP_VISIBLE_DEVICES=1 my_program.exe`。

方式 B：从源码编译。 HIP SDK 7.1.0 的 `clang.exe` **不包含** MSVC 集成。你必须告诉它本地的 Visual Studio BuildTools 和 Windows SDK 路径。以本工具包中的 `dgpu-build-template.ps1` 为模板。

关键参数： `--offload-arch=<你的 dGPU 架构>`。HIP SDK 的 clang 默认 `gfx906`，如果你的 dGPU 不是 `gfx906`，运行时会出现 "device kernel image is invalid"。

阶段 5 — 验证

```
# iGPU (默认 shell, 任何终端)
& C:\rocm-sdk\.venv\Scripts\python.exe -m rocm_sdk test # 26/26 (1 个 Linux 跳过)

# dGPU (activate-dgpu.ps1 之后)
hipInfo # 两块设备
```

<code>\$env:HIP_VISIBLE_DEVICES = '1'; hipInfo</code>	# 仅 dGPU
<code>& C:\rocm-sdk-dgpu\vector_add.exe</code>	# 已编译内核, gfx1100

适配其他硬件

硬件	需要修改什么	保持不变
Strix Halo (Ryzen AI MAX 395) iGPU (gfx1151)	无需修改 — 本工具包在此硬件上验证开发	阶段 1–5 全部适用
Strix Point (HX 370/470) iGPU (gfx1155)	TheRock 索引: <code>gfx1151</code> (尝试) 或 <code>gfx12-generic</code> (如可用) ; 可能需要 <code>-libraries-gfx1151</code>	阶段 1–5 保持不变
Rembrandt / Phoenix (gfx1103) iGPU	TheRock 索引: <code>gfx110X-all</code> ; <code>-libraries-gfx110x-all</code>	阶段 1–5 保持不变
RX 7600 XT (gfx1100) dGPU	<code>--offload-arch=gfx1100</code> ; HIP SDK 7.1.0 已支持	仅阶段 4
RX 9070 / 9070 XT (RDNA 4, gfx1201) dGPU	<code>--offload-arch=gfx1201</code> ; 确认 HIP SDK 版本支持 gfx1201 (7.1.0+ 可能需要更新)	仅阶段 4; 可能需要升级 HIP SDK
RDNA 2 (gfx1031, 如 RX 6600) dGPU	<code>--offload-arch=gfx1031</code> ; 确认 HIP SDK 支持	仅阶段 4
没有 Visual Studio BuildTools	dGPU C++ 编译将无法工作; Python 虚拟环境侧不受影响	阶段 4 失败; 运行时仍可用
没有系统 HIP SDK	dGPU 侧无可回退; 如果你只有 TheRock 轮子覆盖你的 dGPU 目标, 按阶段 1 的方式安装 TheRock dGPU 虚拟环境	不适用

关键约束 / 不变式

- **驱动是 `gcNArchName` 的唯一可信来源。** 在决定方案前, 始终从每个 ROCm 安装运行 `hipInfo` 确认目标。
- **驱动级 `non-peer`:** 即使两块 GPU 都可见, AMD 也会把大多数 iGPU/dGPU 配对标记为 `non-peers` (例如 Strix Halo + RX 7900 XTX, `gfx1151` + `gfx1100`) 。GPU-GPU 直接拷贝不可行。需走主机内存中转。
- **TheRock `sdist` 风格:** `rocm-7.X.tar.gz` 来自 `https://repo.amd.com/rocm/whl/<target>/` 是**目标特定的** (不同索引的 MD5 不同) 。始终从与你的 `libraries` 轮子**相同的**索引下载 `sdist`。
- **轮子索引缓存污染:** 如果 `pip install --find-links` 报 "No such file or directory: ", 删除缓存中残留的 `*_index.html` 文件 — `pip` 会把它们误读为带 `../` 相对路径的 PEP 503 simple 页面。
- **HIP_PATH 覆盖:** HIP SDK 7.1.0 的 `hipconfig` 从环境读 `HIP_PATH` 。如果 iGPU 虚拟环境设置了 `HIP_PATH` , HIP SDK 的 `hipconfig` 会报告 iGPU 虚拟环境的路径。激活脚本必须清空 `HIP_PATH` 并设为 HIP SDK 目录。

- **路径含空格的 `hipcc.bat` 不工作**（它按空格分词）。直接调用 `clang.exe`，使用 `--driver-mode=g++ --hip-link`。

Ollama 双 GPU 加速 (v1.2.0)

本工具包现在包含利用两块 GPU 同时加速本地 LLM 推理的工具。

Non-peers VRAM 测试

运行 `test-peer-vram.ps1` 验证 non-peers 约束和主机内存中转方案：

```
hipDeviceCanAccessPeer(0->1): 0          ← 无直接 peer 访问
hipDeviceCanAccessPeer(1->0): 0          ← 无直接 peer 访问
hipDeviceEnablePeerAccess(1,0): err=101  ← peer 启用失败
hipMemcpyPeer(d0<-d1): err=0 (no error) ← 但拷贝成功
PEER COPY: PASS (transparent host staging by HIP runtime)
STAGING COPY: PASS
```

发现：iGPU 和 dGPU **可以在彼此之间拷贝 VRAM** — 只是不能通过直接 peer-to-peer。HIP SDK 7.1.0 的 `hipMemcpyPeer` 在 peer 访问不可用时透明回退到主机内存中转，数据正确往返。

Ollama 调度器行为

Ollama 的调度器在架构上是**单 GPU 每模型**：

行为	详情
<code>NO_PEER_COPY=1</code>	llama.cpp 检测到 non-peers 时设置
GPU 选择	调度器为每个模型加载选择一块 GPU（ <code>sched.go:1024</code> ）
溢出	当模型超过一块 GPU 的 VRAM 时，溢出到 系统内存（CPU） ，而非另一块 GPU
<code>LLAMA_ARG_SPLIT_MODE=layer</code>	被 llama-server 继承但无效 — runner 不传递 <code>--device 0,1</code>
<code>LLAMA_ARG_DEVICE=0,1</code>	崩溃： <code>invalid device: 0</code> （内置二进制未编译 GPU 支持）

可用方案：双模型双 GPU（方案 1）

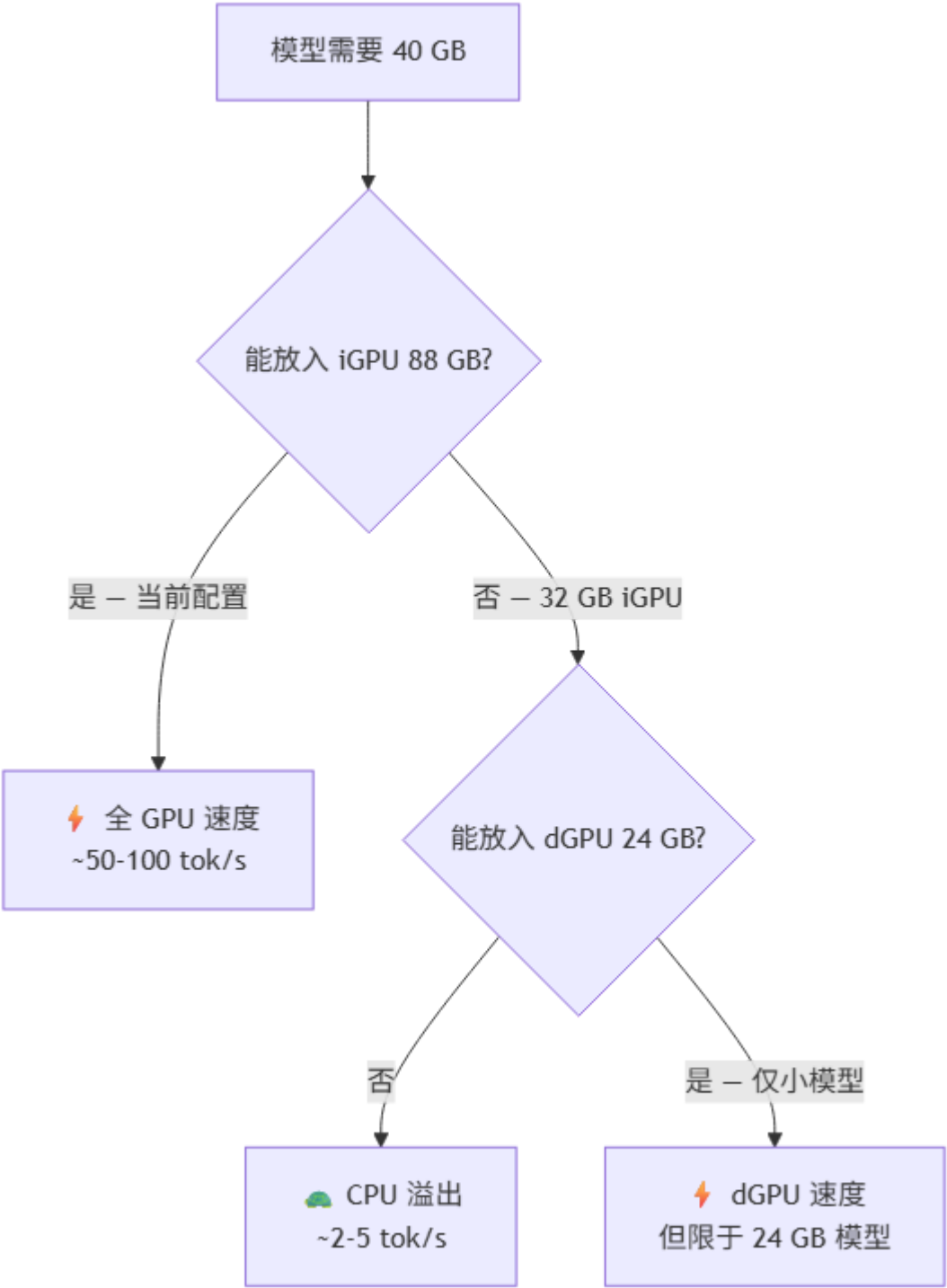
`configure-ollama-dual-gpu.ps1` 设置用户级环境变量并优雅重启 Ollama 托盘应用：

```
.\configure-ollama-dual-gpu.ps1          # 应用配置
.\configure-ollama-dual-gpu.ps1 -Revert  # 恢复默认
```

运行后，加载两个模型： - 大模型（如 `gemma4:26b-a4b-it-q8_0`，约 28 GB）→ 落在 iGPU（87 GB） - 小模型（如 `gemma4:12b-it-q8_0`，约 12 GB）→ 落在 dGPU（24 GB）

对不同模型的并发请求在不同 GPU 上并行运行。

性能瓶颈层级图



层级	带宽	角色
GPU VRAM (iGPU 88 GB / dGPU 24 GB)	~500 GB/s	并行矩阵运算 — 快
系统内存 (64 GB DDR5)	~90 GB/s	CPU 溢出 — 比 GPU 慢 10-50 倍
NPU (Strix Halo 上的 XDNA)	N/A	Ollama/llama.cpp 不使用

建议

工作负载	最佳方案
单个大模型 (≤ 87 GB)	仅用 iGPU (87 GB VRAM 可容纳大多数模型)
多用户 / 多模型	方案 1: 双模型双 GPU (configure-ollama-dual-gpu.ps1)
模型对 iGPU 来说太大	减小上下文 (OLLAMA_CONTEXT_LENGTH=32768) 或使用 Q4 量化

不要为了增加系统内存而减少 iGPU VRAM。iGPU 的 87 GB 统一内存是本机最大的优势。CPU 溢出比 GPU 慢 10-50 倍。NPU (XDNA) 不被 Ollama/llama.cpp 使用。

工具包文件清单

C:\therock\rocm-dual-gpu-kit\	
├─ README.md	<- 英文
├─ README.zh-CN.md	<- 简体中文 (本文件)
├─ README.ja-JP.md	<- 日本語
├─ README.ko-KR.md	<- 한국어
├─ LICENSE	<- Apache License 2.0 (全文)
├─ NOTICE	<- 版权与归属
├─ kit.json	<- 工具包元数据
├─ AGENTS.md	<- Agent 快速入门合约
├─ SKILL.md	<- 结构化技能格式
├─ install-igpu-venv.ps1	<- 阶段 1: 自动检测 + 安装
├─ rewire-igpu.ps1	<- 阶段 2: 机器范围环境 (UAC)
├─ activate-dgpu.ps1	<- 阶段 3: HIP SDK 激活
├─ deactivate-dgpu.ps1	<- 阶段 3: 恢复
├─ dgpu-build-template.ps1	<- 阶段 4: 克隆并按你的 dGPU 定制
├─ detect-hardware.ps1	<- 检测 iGPU/dGPU + 其 gcArchName + HIP SDK
├─ validate.ps1	<- 阶段 5: 端到端冒烟测试
├─ diagnose-connection.ps1	<- 阶段 5.5: 只读传输诊断
├─ dgpu-probe.ps1	<- 可选: 更细粒度的 PnP 探测
├─ peer_vram_test.cpp	<- v1.2.0: HIP C++ non-peers VRAM 测试
├─ test-peer-vram.ps1	<- v1.2.0: 编译 + 运行 peer_vram_test
├─ configure-ollama-dual-gpu.ps1	<- v1.2.0: Ollama 双 GPU 配置 + 托盘重启
├─ start-dual-gpu-ollama.ps1	<- v1.2.0: 独立 Ollama 启动器 (已被取代)
└─ start-split-model.ps1	<- v1.2.0: 强制层分割尝试 (受限)

许可与版权

本工具包 Copyright 2026 cubecloud Limited (<https://cubecloud.io>)，基于 Apache License, Version 2.0 许可发布。

完整许可文本见 [LICENSE](#)，归属与商标信息见 [NOTICE](#)。

为什么选择 Apache 2.0

原因	对 cubecloud 的好处
与 AMD ROCm 和 TheRock 同样的许可	同一生态内不产生许可冲突
显式的专利授权 + 反诉条款	保护我们免受下游用户的专利诉讼
宽松许可：商用、修改、分发、私用	不阻碍 cubecloud 在内部商用使用本工具包
与 GPLv3 兼容	下游如需可在 GPL 下重新许可
硬件/计算工具的事实标准	被企业用户认可和信任
要求保留版权与许可	归属于 cubecloud Limited 的信息在所有副本中持久存在
商标（AMD、Radeon 等）不在授权范围内	我们不出让 AMD 的商标；只是记录如何使用它们

商标 / 归属

- AMD、Radeon、ROCm、HIP、Adrenalin、Strix Halo、RDNA 是 Advanced Micro Devices, Inc. 的商标。
- 本工具包与 AMD 无关联，未获得其认可或赞助。
- "cubecloud" 和 "cubecloud.io" 是 cubecloud Limited 的商标。

第三方组件（本工具包不重新分发）

本工具包不包含 ROCm、TheRock、HIP SDK 或任何 AMD 二进制。它只是一个配置 / 编排层，指向 AMD 官方安装路径和官方 Python 轮子索引。请参考这些组件各自的许可。

组件	许可	来源
AMD ROCm / TheRock	Apache 2.0	https://github.com/ROCm/TheRock
AMD HIP SDK	MIT（按 AMD 安装程序 EULA）	AMD 安装程序
Python 3.12	PSF 许可	https://www.python.org
Visual Studio BuildTools	Microsoft EULA	https://visualstudio.microsoft.com

如果对本工具包进行分叉或在商业场景中复用，请保留 `cubecloud Limited` 版权信息、`LICENSE` 文件和 `NOTICE` 文件。